1) Count the occurrence of each number (frequency map)

Interview Question

Given a list of integers, count how many times each number appears using Java Streams.

Example Input

```
List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4, 4, 9, 7, 2);
```

Detailed Explanation

This is one of the **most important logical stream questions** because it tests:

- `Collectors.groupingBy()`
- `Collectors.counting()`
- understanding of frequency maps

We group elements by their value, then count how many times each value appears.

Stream Solution

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

public class CountOccurrences {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4, 4, 9, 7, 2);

        Map<Integer, Long> frequencyMap = numbers.stream()
                .collect(Collectors.groupingBy(
                        Function.identity(),
                        Collectors.counting()
                ));
```

```
        System.out.println(frequencyMap);
    }
}
```

Output

```
{2=3, 4=3, 7=2, 9=1}
```

Interviewer Follow-up

Q: Why do we use `Function.identity()` here?
A: It returns the element itself as the grouping key.

2) Print only duplicate numbers

Interview Question

Given a list of integers, print only the duplicate numbers using Java Streams.

Example Input

```
List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4, 4, 9, 7, 2);
```

Detailed Explanation

A duplicate number means:

- Frequency > 1

So the logic is:

1. Build a frequency map
2. Filter entries where count > 1
3. Extract keys

This is a very common interview question because it combines:

- grouping

- filtering
- mapping

Stream Solution

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

public class FindDuplicates {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4,
4, 9, 7, 2);

        List<Integer> duplicates = numbers.stream()
                .collect(Collectors.groupingBy(
                        Function.identity(),
                        Collectors.counting()
                ))
                .entrySet()
                .stream()
                .filter(entry -> entry.getValue() > 1)
                .map(Map.Entry::getKey)
                .sorted()
                .collect(Collectors.toList());

        System.out.println("Duplicate numbers: " +
duplicates);
    }
}
```

Output

```
Duplicate numbers: [2, 4, 7]
```

Interviewer Follow-up

Q: How can you preserve insertion order of duplicates?
A: Use `LinkedHashMap` in `groupingBy()` or use a `Set`-based approach with `filter(n -> !seen.add(n))`.

3) Find the highest duplicate number

Interview Question

Given a list of integers, find the highest number that appears more than once using Java Streams.

Example Input

```
List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4, 4, 9, 7, 2, 10, 10);
```

Detailed Explanation

We need:

1. Count frequencies
2. Keep only duplicates (count > 1)
3. Extract keys
4. Find max

This is a nice interview question because it tests:

- frequency logic
- filtering
- `max()`

Stream Solution

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

public class HighestDuplicate {
```

```
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4,
4, 9, 7, 2, 10, 10);

        Optional<Integer> highestDuplicate = numbers.stream()
                .collect(Collectors.groupingBy(
                        Function.identity(),
                        Collectors.counting()
                ))
                .entrySet()
                .stream()
                .filter(entry -> entry.getValue() > 1)
                .map(Map.Entry::getKey)
                .max(Integer::compareTo);

        System.out.println("Highest duplicate number: " +
highestDuplicate.orElse(null));
    }
}
```

Output

```
Highest duplicate number: 10
```

Interviewer Follow-up

Q: What if no duplicate exists?
A: `max()` returns `Optional.empty()`, so we should handle it safely using `orElse()` or `orElseThrow()`.

4) Find the second highest duplicate number

Interview Question

Given a list of integers, find the second highest number that appears more than once using Java Streams.

Example Input

```
List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4, 4, 9, 7,
2, 10, 10, 8, 8);
```

Detailed Explanation

This is a very good interview-level logical question.

Steps:

1. Count frequencies
2. Keep only duplicates
3. Extract duplicate numbers
4. Sort in descending order
5. Skip the first (highest)
6. Get the next one (second highest)

This tests:

- grouping
- filtering
- sorting
- `skip()`
- `findFirst()`

Stream Solution

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

public class SecondHighestDuplicate {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4,
4, 9, 7, 2, 10, 10, 8, 8);

        Optional<Integer> secondHighestDuplicate =
numbers.stream()
```

```
                .collect(Collectors.groupingBy(
                        Function.identity(),
                        Collectors.counting()
                ))
                .entrySet()
                .stream()
                .filter(entry -> entry.getValue() > 1)
                .map(Map.Entry::getKey)
                .sorted(Comparator.reverseOrder())
                .skip(1)
                .findFirst();

        System.out.println("Second highest duplicate number:
" + secondHighestDuplicate.orElse(null));
    }
}
```

Output

```
Second highest duplicate number: 8
```

Interviewer Follow-up

Q: What if only one duplicate number exists?
A: Then after `skip(1)`, the stream becomes empty, so `findFirst()` returns `Optional.empty()`.

5) Remove duplicate numbers and return unique numbers

Interview Question

Given a list of integers, remove duplicates and return only unique numbers using Java Streams.

Example Input

```
List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4, 4, 9, 7,
2);
```

Detailed Explanation

This is one of the simplest but very common stream questions.

Java Streams provides:

- `distinct()`

It removes repeated values while preserving the encounter order (for ordered streams like `List`).

This is frequently asked as a quick warm-up question.

Stream Solution

```
import java.util.*;
import java.util.stream.Collectors;

public class RemoveDuplicates {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4,
4, 9, 7, 2);

        List<Integer> uniqueNumbers = numbers.stream()
                .distinct()
                .collect(Collectors.toList());

        System.out.println("Unique numbers: " +
uniqueNumbers);
    }
}
```

Output

```
Unique numbers: [4, 2, 7, 9]
```

Interviewer Follow-up

Q: How does `distinct()` work internally?
A: It uses `equals()` and `hashCode()` to identify unique elements (for objects, correct `equals/hashCode` is important).

) Find the first non-repeated number

Interview Question

Given a list of integers, find the first number that appears only once using Java Streams.

Example Input

```
List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4, 9, 7,
10, 11, 10);
```

Detailed Explanation

This is a **very famous interview question**.

We need:

1. Count frequency of each number
2. Preserve insertion order
3. Find the first number whose count is `1`

Important:

If we use normal `HashMap`, insertion order is not guaranteed.
So we should use **`LinkedHashMap`**.

Stream Solution

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

public class FirstNonRepeated {
```

```
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4,
9, 7, 10, 11, 10);

        Optional<Integer> firstNonRepeated = numbers.stream()
                .collect(Collectors.groupingBy(
                        Function.identity(),
                        LinkedHashMap::new,
                        Collectors.counting()
                ))
                .entrySet()
                .stream()
                .filter(entry -> entry.getValue() == 1)
                .map(Map.Entry::getKey)
                .findFirst();

        System.out.println("First non-repeated number: " +
firstNonRepeated.orElse(null));
    }
}
```

Output

```
First non-repeated number: 9
```

Interviewer Cross-Question

Q: Why do we use `LinkedHashMap` instead of `HashMap`?
A: Because `LinkedHashMap` preserves insertion order, which is necessary for "first" non-repeated element.

2) Find the second most repeated number

Interview Question

Given a list of integers, find the second most repeated number using Java Streams.

Example Input

```
List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4, 4, 9, 7,
2, 10, 10, 10, 8, 8);
```

Detailed Explanation

This is a **strong interview-level frequency problem**.

Steps:

1. Count frequency of each number
2. Sort entries by frequency descending
3. Skip the first (most repeated)
4. Return the next key

Frequency:

- 10 → 3 times
- 2 → 3 times
- 4 → 3 times
- 7 → 2 times
- 8 → 2 times
- 9 → 1 time

⚠️ Important note:
If multiple numbers have same frequency, result depends on sort tie handling.
In interviews, mention this clearly.

Stream Solution

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

public class SecondMostRepeated {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4,
4, 9, 7, 2, 10, 10, 10, 8, 8);
```

```
        Optional<Integer> secondMostRepeated =
numbers.stream()
                .collect(Collectors.groupingBy(
                        Function.identity(),
                        Collectors.counting()
                ))
                .entrySet()
                .stream()
                .sorted((e1, e2) ->
Long.compare(e2.getValue(), e1.getValue()))
                .skip(1)
                .map(Map.Entry::getKey)
                .findFirst();

        System.out.println("Second most repeated number: " +
secondMostRepeated.orElse(null));
    }
}
```

Output

```
Second most repeated number: 2
```

Interviewer Cross-Question

Q: What if multiple numbers have the same highest frequency?
A: Then the result depends on tie-breaking. In real interviews, clarify whether they want "second entry after sorting" or "second distinct frequency".

🔥 This is a **very smart interview clarification**.

3) Find top 2 most repeated numbers

Interview Question

Given a list of integers, find the top 2 most repeated numbers using Java Streams.

Example Input

```
List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4, 4, 9, 7,
2, 10, 10, 10, 8, 8);
```

Detailed Explanation

This question tests:

- frequency map
- sorting by frequency
- limiting results

Steps:

1. Count frequencies
2. Sort descending by count
3. Limit 2
4. Return keys

Stream Solution

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

public class Top2MostRepeated {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4,
4, 9, 7, 2, 10, 10, 10, 8, 8);

        List<Integer> top2 = numbers.stream()
                .collect(Collectors.groupingBy(
                        Function.identity(),
                        Collectors.counting()
                ))
                .entrySet()
                .stream()
                .sorted((e1, e2) ->
Long.compare(e2.getValue(), e1.getValue()))
                .limit(2)
                .map(Map.Entry::getKey)
```

```
                .collect(Collectors.toList());

        System.out.println("Top 2 most repeated numbers: " +
top2);
    }
}
```

Output

```
Top 2 most repeated numbers: [2, 4]
```

Interviewer Cross-Question

Q: Can we return both number and count instead of only numbers?
A: Yes, return the `Map.Entry` list or map it into a custom DTO.

4) Group even and odd numbers using Streams

Interview Question

Given a list of integers, separate even and odd numbers using Java Streams.

Example Input

```
List<Integer> numbers = Arrays.asList(4, 2, 7, 9, 10, 11, 14,
15);
```

Detailed Explanation

This is a classic but slightly more advanced collector question.

Instead of manually filtering twice, we can use:

- `Collectors.partitioningBy()`

This is ideal when the condition results in **two groups only**:

- true

- false

Here:

- `true` → even
- `false` → odd

Stream Solution

```
import java.util.*;
import java.util.stream.Collectors;

public class EvenOddPartition {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(4, 2, 7, 9, 10,
11, 14, 15);

        Map<Boolean, List<Integer>> partitioned =
numbers.stream()
                .collect(Collectors.partitioningBy(n -> n % 2
== 0));

        System.out.println("Even numbers: " +
partitioned.get(true));
        System.out.println("Odd numbers: " +
partitioned.get(false));
    }
}
```

Output

```
Even numbers: [4, 2, 10, 14]
Odd numbers: [7, 9, 11, 15]
```

Interviewer Cross-Question

Q: What is the difference between `groupingBy()` and `partitioningBy()`?
A:

- `partitioningBy()` is specifically for **boolean conditions** (2 groups only)

- `groupingBy()` is for **general grouping** (multiple groups)

5) Sort numbers by frequency (highest repeated first)

Interview Question

Given a list of integers, sort the unique numbers based on their frequency in descending order using Java Streams.

Example Input

```
List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4, 4, 9, 7, 2, 10, 10, 10, 8, 8);
```

Detailed Explanation

This is a very good interview problem because it combines:

- frequency counting
- sorting map entries by value
- extracting keys

Steps:

1. Build frequency map
2. Convert map entries to stream
3. Sort descending by count
4. Extract keys

Stream Solution

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;
```

```
public class SortByFrequency {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4,
4, 9, 7, 2, 10, 10, 10, 8, 8);

        List<Integer> sortedByFrequency = numbers.stream()
                .collect(Collectors.groupingBy(
                        Function.identity(),
                        Collectors.counting()
                ))
                .entrySet()
                .stream()
                .sorted((e1, e2) ->
Long.compare(e2.getValue(), e1.getValue()))
                .map(Map.Entry::getKey)
                .collect(Collectors.toList());

        System.out.println("Numbers sorted by frequency: " +
sortedByFrequency);
    }
}
```

Output

```
Numbers sorted by frequency: [2, 4, 10, 7, 8, 9]
```

Interviewer Cross-Question

Q: Can we sort by frequency descending and then by number ascending for tie-breaking?
A: Yes, use comparator chaining.

Example:

```
.sorted(
    Comparator.<Map.Entry<Integer,
Long>>comparingLong(Map.Entry::getValue)
        .reversed()
        .thenComparing(Map.Entry::getKey)
)
```

🔥 This is a **very strong interview answer**.

🔥 Bonus Advanced Question — Find duplicate numbers with their count

This is very useful because interviewers often ask:

“Don’t just print duplicates — print duplicates with frequency.”

Question

Find all duplicate numbers and their counts.

Solution

```
import java.util.*;
import java.util.function.Function;
import java.util.stream.Collectors;

public class DuplicatesWithCount {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(4, 2, 7, 2, 4,
4, 9, 7, 2);

        Map<Integer, Long> duplicatesWithCount =
numbers.stream()
                .collect(Collectors.groupingBy(
                        Function.identity(),
                        Collectors.counting()
                ))
                .entrySet()
                .stream()
                .filter(entry -> entry.getValue() > 1)
                .collect(Collectors.toMap(
                        Map.Entry::getKey,
                        Map.Entry::getValue
                ));

        System.out.println("Duplicates with count: " +
duplicatesWithCount);
    }
}
```

Output

```
Duplicates with count: {2=3, 4=3, 7=2}
```

1. **Group employees by department**
2. **Highest salary employee**

I'll give you both in your usual interview-prep style:

- **Question**
- **Detailed Explanation**
- **Code**
- **Output**
- **Interviewer Cross-question**

Common `Employee` Class for Both Questions

```
import java.util.*;

class Employee {
    private int id;
    private String name;
    private String department;
    private double salary;

    public Employee(int id, String name, String department,
double salary) {
        this.id = id;
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    public int getId() {
        return id;
    }
```

```
    public String getName() {
        return name;
    }

    public String getDepartment() {
        return department;
    }

    public double getSalary() {
        return salary;
    }

    @Override
    public String toString() {
        return "Employee{id=" + id +
                ", name='" + name + '\'' +
                ", department='" + department + '\'' +
                ", salary=" + salary +
                '}';
    }
}
```

Sample Data

```
List<Employee> employees = Arrays.asList(
        new Employee(1, "Murali", "IT", 75000),
        new Employee(2, "Arun", "HR", 50000),
        new Employee(3, "Priya", "IT", 90000),
        new Employee(4, "Kumar", "Finance", 65000),
        new Employee(5, "Divya", "HR", 80000),
        new Employee(6, "Suresh", "Finance", 95000)
);
```

1) Group Employees by Department

Interview Question

How do you group employees by department using Java Streams?

Detailed Explanation

This is one of the **most common Java 8 stream interview questions**.

We want to transform:

```
List<Employee>
```

into:

```
Map<String, List<Employee>>
```

Where:

- Key = department name
- Value = list of employees in that department

Best Stream API:

- `Collectors.groupingBy()`

Logic:

1. Stream over employees
2. Use department as grouping key
3. Collect into a map

Stream Solution

```
import java.util.*;
import java.util.stream.Collectors;

public class GroupByDepartmentExample {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
                new Employee(1, "Murali", "IT", 75000),
                new Employee(2, "Arun", "HR", 50000),
                new Employee(3, "Priya", "IT", 90000),
                new Employee(4, "Kumar", "Finance", 65000),
                new Employee(5, "Divya", "HR", 80000),
                new Employee(6, "Suresh", "Finance", 95000)
        );
```

```
        Map<String, List<Employee>> groupedByDepartment =
employees.stream()
                .collect(Collectors.groupingBy(Employee::getD
epartment));

        groupedByDepartment.forEach((dept, empList) -> {
            System.out.println(dept + " -> " + empList);
        });
    }
}
```

Output

```
IT -> [Employee{id=1, name='Murali', department='IT',
salary=75000.0}, Employee{id=3, name='Priya',
department='IT', salary=90000.0}]
HR -> [Employee{id=2, name='Arun', department='HR',
salary=50000.0}, Employee{id=5, name='Divya',
department='HR', salary=80000.0}]
Finance -> [Employee{id=4, name='Kumar',
department='Finance', salary=65000.0}, Employee{id=6,
name='Suresh', department='Finance', salary=95000.0}]
```

Simple Explanation

`groupingBy(Employee::getDepartment)` means:

"Put employees into buckets based on department."

Interviewer Cross-Question

Q: How do you count employees in each department instead of grouping full objects?
A: Use `Collectors.counting()` with `groupingBy()`.

Example:

```
Map<String, Long> countByDepartment = employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.counting()
```

```
        ));
```

2) Find the Highest Salary Employee

Interview Question

How do you find the employee with the highest salary using Java Streams?

Detailed Explanation

This is another **top interview question**.

We need:

1. Stream the employee list
2. Compare employees by salary
3. Get the maximum one

Best Stream API:

- `max()`
- `Comparator.comparingDouble()`

Why `Optional<Employee>`?

Because the list may be empty, so `max()` safely returns `Optional`.

Stream Solution

```
import java.util.*;
import java.util.Comparator;

public class HighestSalaryEmployeeExample {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
                new Employee(1, "Murali", "IT", 75000),
                new Employee(2, "Arun", "HR", 50000),
                new Employee(3, "Priya", "IT", 90000),
                new Employee(4, "Kumar", "Finance", 65000),
```

```
                new Employee(5, "Divya", "HR", 80000),
                new Employee(6, "Suresh", "Finance", 95000)
        );

        Optional<Employee> highestSalaryEmployee =
employees.stream()
                .max(Comparator.comparingDouble(Employee::get
Salary));

        System.out.println("Highest salary employee: " +
highestSalaryEmployee.orElse(null));
    }
}
```

Output

```
Highest salary employee: Employee{id=6, name='Suresh',
department='Finance', salary=95000.0}
```

Simple Explanation

This logic means:

“Compare all employees by salary and return the one with the highest salary.”

Interviewer Cross-Question

Q: How do you get only the highest salary value instead of the whole employee object?
A: Use `map(Employee::getSalary)` before `max()`.

Example:

```
OptionalDouble highestSalary = employees.stream()
        .mapToDouble(Employee::getSalary)
        .max();

System.out.println(highestSalary.orElse(0));
```

🔥 Bonus (Very Important) — Highest Salary Employee in Each Department

This is a **very common advanced interview follow-up** after your two questions.

Interview Question

How do you find the highest salary employee in each department using Java Streams?

Stream Solution

```
import java.util.*;
import java.util.stream.Collectors;
import static java.util.stream.Collectors.*;

public class HighestSalaryByDepartment {
    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
                new Employee(1, "Murali", "IT", 75000),
                new Employee(2, "Arun", "HR", 50000),
                new Employee(3, "Priya", "IT", 90000),
                new Employee(4, "Kumar", "Finance", 65000),
                new Employee(5, "Divya", "HR", 80000),
                new Employee(6, "Suresh", "Finance", 95000)
        );

        Map<String, Optional<Employee>> highestByDepartment =
employees.stream()
                .collect(groupingBy(
                        Employee::getDepartment,

maxBy(Comparator.comparingDouble(Employee::getSalary))
                ));

        highestByDepartment.forEach((dept, emp) ->
                System.out.println(dept + " -> " +
emp.orElse(null))
        );
    }
}
```

Output

```
IT -> Employee{id=3, name='Priya', department='IT',
salary=90000.0}
HR -> Employee{id=5, name='Divya', department='HR',
salary=80000.0}
Finance -> Employee{id=6, name='Suresh',
department='Finance', salary=95000.0}
```

Why this is powerful

This uses:

- `groupingBy()`
- downstream collector
- `maxBy()`

🔥 This is a **very strong Java 8 interview answer**.

Quick Revision (For Interview)

1. Group employees by department

```
Map<String, List<Employee>> grouped = employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartmen
t));
```

2. Highest salary employee

```
Optional<Employee> highest = employees.stream()
        .max(Comparator.comparingDouble(Employee::getSalary))
;
```

3. Highest salary employee in each department

```
Map<String, Optional<Employee>> result = employees.stream()
        .collect(Collectors.groupingBy(
                Employee::getDepartment,

Collectors.maxBy(Comparator.comparingDouble(Employee::getSala
ry))
        ));
```